MCMP117

Fundamentals of Operating Systems

Week 1 · The Big Picture & a History of Computing

What is a computer? What is an operating system? And how did we get here?

Electronical Communication Technology Programme · Vocational School

Welcome

INSTRUCTOR

Asst. Prof. Dr. Hasan Oğuz, Ph.D.

Head of the Electronical Communication Technology Programme

Slides, lecture notes and example code for every week are in the course repository.

HOW THE SEMESTER RUNS

Weeks 1–7: how an operating system works (theory, small C experiments)

Week 8: midterm exam

Weeks 9–14: hands-on Linux and the Bash shell

Week 15: final exam

Today's Agenda

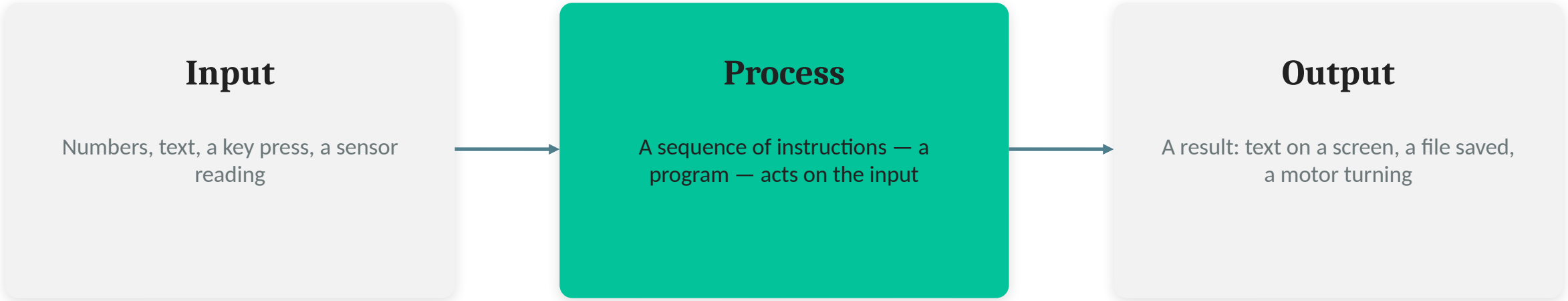
1. What is a computer, really?
2. What is an operating system? (a first, simple definition)
3. Why do we need one? — the purpose
4. An extended history — from punch cards to the cloud
5. Why this history matters

Next week: formal definitions, OS goals, and how operating systems are structured.

THE BIG PICTURE

What Is a Computer, Really?

Strip away the screen, the keyboard, the apps — at its core, a computer is a machine that follows instructions.



Every computer you will ever meet — a phone, a laptop, a drone flight controller — is this same loop, over and over, billions of times a second.

THE BIG PICTURE

What Is an Operating System?

A FIRST DEFINITION (WE'LL SHARPEN THIS NEXT WEEK)

The operating system is the program that starts first when you turn the computer on, and manages everything else — so that other programs can run.

It starts first

The very first program to run when you power on a computer.

It manages everything else

Other programs — your browser, a game, a text editor — run with its help.

It hides the hard parts

You don't need to know how the disk or memory works to save a file.

THE BIG PICTURE

Why Do We Need an Operating System? — The Purpose

Without an OS

Only one program could use the computer at a time.

That program would have to talk to the hardware directly — the disk, the screen, the keyboard — which is hard to get right and easy to break.

This is roughly how the very first computers worked, as we'll see shortly.

With an OS

Many programs can safely share the same hardware.

You don't need to understand the disk or the memory to use a program — the OS handles it.

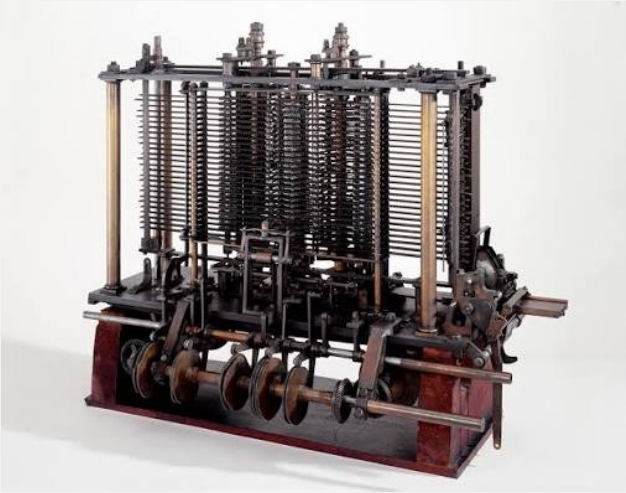
Multiple people, or multiple programs, can use the same machine at once.

HISTORY

How Did We Get Here?

From a machine with no software at all, to the operating system in your pocket.

Before There Were Computers, There Were Calculators



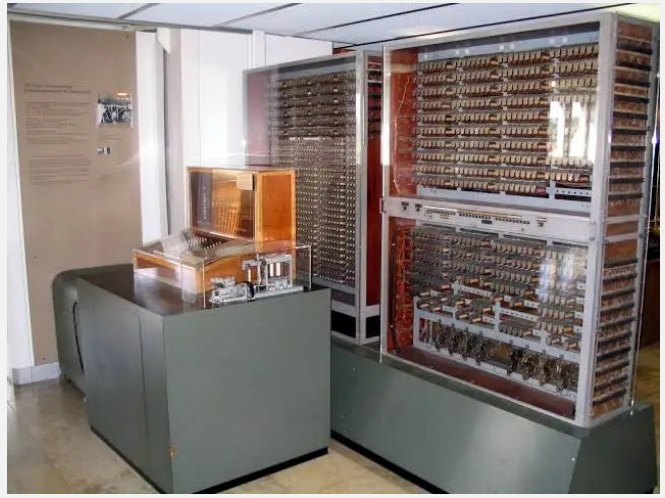
1837–43 · Analytical Engine

Charles Babbage designs a mechanical, general-purpose computer with a processor (“mill”) and memory (“store”), fed by punched cards. Ada Lovelace publishes the first program for it in 1843. Never fully built — shown is Babbage's trial model of the mill.



1890 · Hollerith Tabulator

Herman Hollerith's punch-card machine counts the U.S. Census: pins pass through the holes, close an electrical circuit, and advance a dial. Years of manual counting shrink to months; his company later becomes part of IBM.



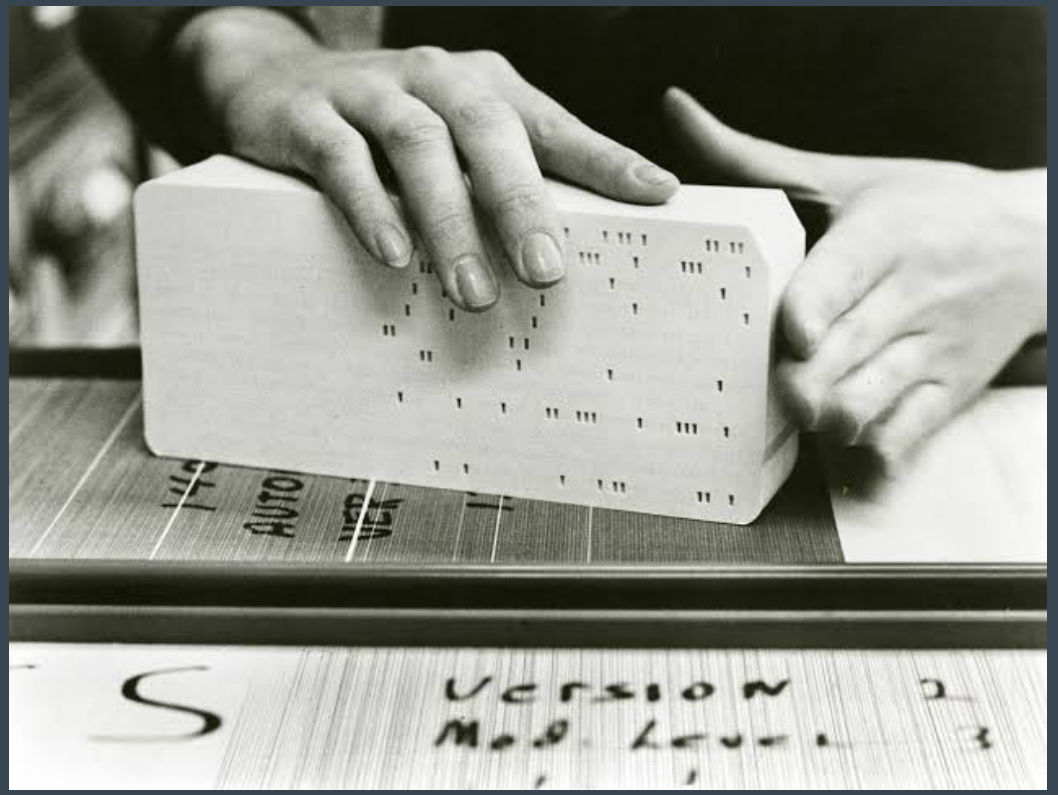
1936–41 · Zuse Z1–Z3

Konrad Zuse builds a series of programmable machines in Berlin. The relay-based Z3 (1941) is among the first working programmable computers. The original was destroyed in 1943 — shown is a later museum replica.

Punch Cards — Reading Data by the Hole

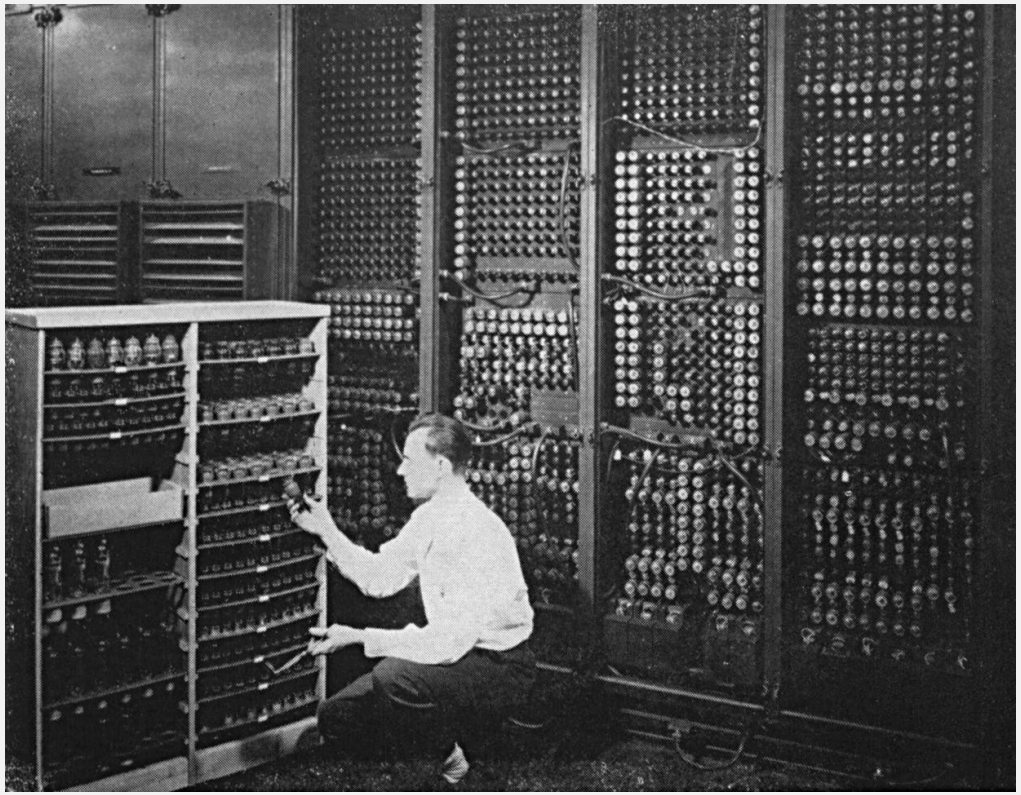
HOW IT ACTUALLY WORKED

- A punch card is a stiff paper card where the presence or absence of a small rectangular hole, at a specific row and column, encodes one character or digit.
- Hollerith's 1890 census machine read holes mechanically, using spring-loaded pins that completed an electrical circuit through each hole.
- By the 1950s–60s, punch cards carried both data and program instructions — one card per line of code was completely normal.
- A dropped, out-of-order deck of cards was a real, career-defining disaster for a 1960s programmer — one motivation for later, friendlier interfaces.



A punch card being read on a card-punch/verifier machine.

ENIAC — Room-Sized, and Utterly Alone



ENIAC (Electronic Numerical Integrator and Computer), 1945.

THE MACHINE WITH NO OS AT ALL

- Filled a 9m × 15m room and weighed about 30 tons; built at the University of Pennsylvania for the U.S. Army.
- Used roughly 18,000 vacuum tubes instead of any transistor — tubes failed constantly and had to be hunted down by hand.
- No monitor, no keyboard: “programming” meant physically rewiring plugboards and setting switches, which could take days.
- One job ran at a time, with total control of the machine — there is no concept of “operating system” yet, because there is no other software to manage.

GENERATION 1 · 1956

GM-NAA I/O — Arguably the First Operating System

Written jointly by General Motors Research and North American Aviation for GM's IBM 704 mainframe, GM-NAA I/O automated the tedious, error-prone job of loading one program after another by hand.

The problem it solved

Every time one job finished on the IBM 704, an operator had to manually mount tapes and start the next one — the expensive computer sat idle in between.

The idea

Keep a small, permanent “monitor” program resident in memory, whose only job is to automatically load and run the next job in a queue.

Why it counts as “first”

It is the earliest widely-cited program built specifically to manage other programs — the defining trait of every OS since.

GENERATION 1–2

Batch Processing, Step by Step

The dominant style of computing from the 1950s into the 1970s — jobs, not people, are what the computer sees.

1. Punch the cards

A programmer writes a job on a deck of punch cards — there is no typing directly into the machine.

2. Collect a batch

An operator collects many jobs from many people into one "batch" and loads them together.

3. Run automatically

The monitor program runs the whole batch, job after job, with no interaction in between.

4. Collect the printout

Hours (or a day) later, the programmer collects a paper printout — and finds out then whether there was a typo.

This is also why early programmers were so disciplined about checking their work by hand — a single mistake could cost an entire day's turnaround.

GENERATION 2 · 1964

IBM System/360 — One Operating System, Many Machines

The old problem

Before 1964, each IBM computer model needed its own, incompatible software — upgrading meant rewriting everything.

IBM's bet

Design one instruction-set architecture, System/360, spanning small machines to giant mainframes, all running the same family of software.

OS/360

IBM's operating system for the line — one of the largest software projects attempted up to that point, and famously over budget and late.

Why it mattered

It proved that an OS could be a product line of its own — portable across hardware, not welded to one physical machine.

GENERATION 2 · CONCEPT

Multiprogramming — Letting the CPU Multitask

The core insight: while one job waits for slow input/output (a tape, a disk, a printer), the CPU sits idle. Why not let another job use it in the meantime?

Before

One job holds the CPU from start to finish — even while it waits, doing nothing, for a tape to rewind.

The fix

Keep several jobs in memory at once. The moment one starts waiting on I/O, the OS switches the CPU to a job that has real work to do.

The payoff

Total useful work per hour rises sharply — the same expensive mainframe gets far more done without any faster hardware.

GENERATION 2–3 · CONCEPT

Time-Sharing — Many Users, One Machine, At Once

The idea

Instead of one batch job at a time, connect many terminals to one computer. The OS gives each user a tiny slice of CPU time, in rapid rotation.

Why it feels instant

Slices are milliseconds long and rotate dozens of times per second — far faster than a human can perceive, so each user feels like they have the whole machine.

Where it led

Time-sharing systems (CTSS at MIT, then Multics, then UNIX) are the direct ancestors of every multi-user and multi-process system running today, phones included.

This is precisely the mechanism behind the CPU scheduling algorithms you will study in Weeks 4–5 (FCFS, SJF, Priority, Round Robin) — time-sharing is what makes a scheduling algorithm necessary in the first place.

INTERMISSION

10-Minute Break

Next: Multics, UNIX, and the road from the personal computer to the cloud.

GENERATION 2 · 1965–1969

Multics — The Giant That Didn't Ship (On Time)

MIT, Bell Labs, and General Electric set out to build Multics: a single time-sharing system meant to serve a whole city like a public utility — always on, endlessly expandable, deeply secure.

What it promised

Continuous operation, dynamic memory, hierarchical files and directories, and fine-grained security — genuinely ahead of its time.

What went wrong

The design was so ambitious that it grew slower and more expensive than planned; Bell Labs pulled out of the project in 1969.

What it left behind

Two Bell Labs researchers who had worked on it — Ken Thompson and Dennis Ritchie — kept many of its best ideas, and built something smaller instead.

UNIX Is Born From Multics' Ashes

1969

Ken Thompson writes the first version of UNIX at Bell Labs, on a spare PDP-7 minicomputer nobody else wanted — deliberately small and simple, the opposite of Multics.

1970–72

Thompson and Dennis Ritchie grow it into a small set of powerful, composable tools that do one job well and can be chained together.

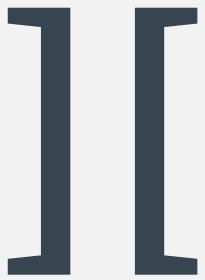
1973

UNIX is rewritten in a new, portable language Ritchie designs for the purpose: C. This is the single decision that lets UNIX later run on almost any hardware.

The philosophy

“Write programs that do one thing and do it well. Write programs to work together.” This idea still shapes every Linux command you will type in Week 9.

Apple II — The Home Computer Goes Mainstream



APPLE][

1977

- Designed by Steve Wozniak, sold by Apple Computer (Jobs & Wozniak) starting in 1977 — one of the first successful, mass-produced personal computers.
- Shipped with color graphics and a built-in version of the BASIC programming language — ready to program the moment it was switched on.
- Its open architecture (expansion slots anyone could build hardware for) created an entire third-party industry around one machine.
- Popularized the idea that a computer could sit on a desk at home or in a classroom, not just in a company's machine room.

GENERATION 4 · 1981

The IBM PC and MS-DOS

ONE PERSON, ONE COMPUTER

- One person, one computer, on a desk — the arrangement this course, and almost every office since, still takes for granted.
- Ran MS-DOS from a removable floppy disk — no hard drive was required to get started.
- Expandable through internal slots, which is part of why the IBM PC "platform" outlived any single model of it.
- 1991, a decade later — on hardware like this, in Helsinki, Linus Torvalds starts writing Linux.



An IBM Personal Computer (5150), with monochrome/color display, 1981.

GENERATION 4 · 1970S–1990S

Graphical Interfaces Arrive — Icons, Windows, a Mouse

Xerox PARC (1973)

Researchers build the Alto, the first computer with overlapping windows, icons, and a mouse — mostly a research machine, never sold widely.

Apple Macintosh (1984)

Apple brings PARC's ideas to a mass-market product: point-and-click, a trash can icon, real desktop metaphors, all with no typed commands required.

Microsoft Windows (1985–95)

Microsoft layers a graphical shell on top of MS-DOS, refining it across versions until Windows 95 makes it the default way most people use a PC.

The lasting shift

For most users, “typing commands to the computer” stops being the norm — though, as we will see in Weeks 9–14, it never disappears underneath the surface.

GENERATION 4–5 · 1991

Linux Is Born

> “I'm doing a (free) operating system (just a hobby, won't be big and professional like GNU)” — Linus Torvalds, a 21-year-old student in Helsinki, in a 1991 Usenet post announcing his kernel.

What it was

A new kernel — the core of an operating system — written from scratch but deliberately UNIX-compatible in behavior and commands.

Why it spread

Released under the GNU General Public License: source code freely available to read, modify, and redistribute, by anyone, forever.

Combined with GNU

Paired with the GNU Project's existing free command-line tools (started by Richard Stallman in 1983), Linux plus GNU became a complete, free operating system.

Meanwhile: Unix and Linux Take the Server Room

While Windows won the desktop, a different, quieter competition was happening on machines most people never see: servers and supercomputers.

Stability & multi-user design

Unix (and later Linux) was built from the start for many simultaneous users and long uninterrupted uptime — exactly what a server needs.

Cost

Linux is free to license, unlike commercial Unix or Windows Server — a decisive advantage once server farms grew into the thousands of machines.

The result

By the 2000s, Linux runs the overwhelming majority of the TOP500 supercomputers and a large share of the servers powering the internet.

GENERATION 6 · 2007

The iPhone — A Phone Becomes a Computer

Apple releases the iPhone in 2007, running a full, modified version of its desktop operating system (what becomes iOS) — not a stripped-down phone OS.

What was new

A capacitive multi-touch screen (no stylus), a real mobile web browser, and — from 2008 — an App Store for installing new software.

Why it's an OS milestone

It proved a full, modern, multi-tasking operating system could run on a battery-powered device that fits in a pocket.

The ripple effect

Every major phone maker follows within two years — this is the direct trigger for the next slide.

GENERATION 6 · 2008

Android — Linux Goes Into Your Pocket

Origins

Android Inc. is founded in 2003, bought by Google in 2005; the first commercial Android phone (the HTC Dream / T-Mobile G1) ships in 2008.

Built on Linux

Android's core is a modified Linux kernel — the same lineage from Slide 22 — wrapped in a new interface, app framework, and Java-based tools.

The open strategy

Unlike iOS, Android is licensed to any phone manufacturer for free, which is why it now runs on the largest number of devices worldwide.

Why it belongs in this course

Every Android phone in this room is, underneath the app icons, running the same kernel family you will type commands into in Week 9.

GENERATION 7 · 2000S

Cloud Computing — One Machine, Acting Like Many

Server hardware kept getting more powerful, but most programs did not need a whole physical machine to themselves. Virtualization answers the mismatch.

Virtual machines

A hypervisor lets one physical server pretend to be several independent computers, each running its own guest operating system, isolated from the others.

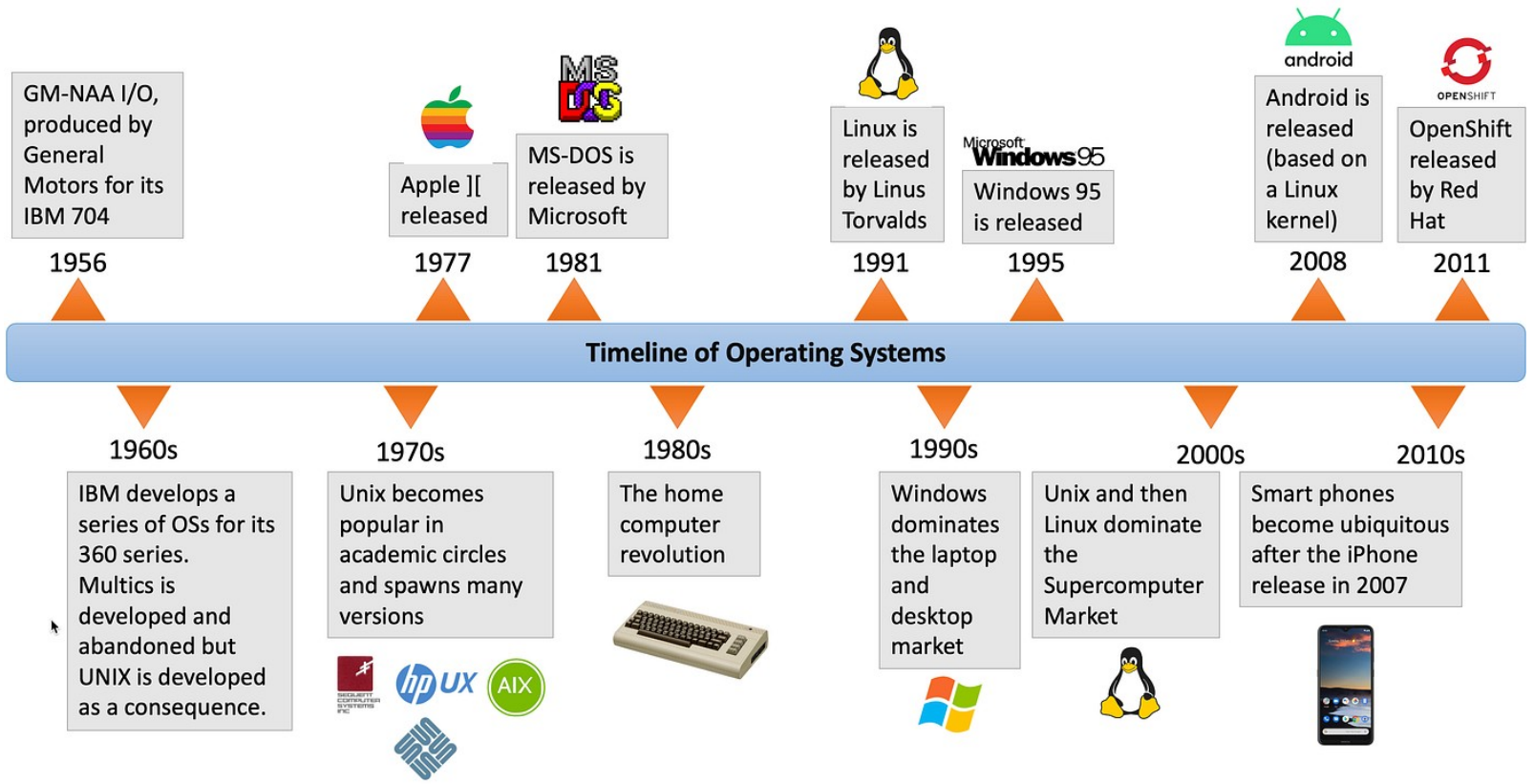
“The cloud”

Companies like Amazon (AWS, 2006) rent out virtual machines by the hour — you use someone else's data center as if it were your own computer.

Why it matters here

This is the same isolation idea behind the containers used for software development and deployment, which is where container platforms such as Kubernetes and OpenShift (2011) come in.

The Full Timeline, at a Glance



Every era from this deck, on one line — from IBM's earliest OS work in the 1950s to the container platforms of the 2010s.

SYNTHESIS

Why This History Matters

Every generation solved a real problem

No OS → automation → sharing the CPU → sharing it interactively → sharing it across the world.

Old ideas never really disappear

Your phone still multiprograms, still time-shares, still batches background tasks.

This course follows the same order

We'll study processes, scheduling, memory, and files roughly in the order they were invented to solve these problems.

WRAP-UP

Week 1 — Key Takeaways

- A computer, at its core, just follows instructions: input → process → output.
- A first working definition: the OS is the program that starts first and manages everything else.
- We need an OS so programs can safely share hardware, and so users don’t need to manage it themselves.
- History moved from no software (ENIAC) → batch → multiprogramming/time-sharing → personal computers → mobile/cloud.
- Two dates worth remembering: UNIX, 1969; Linux, 1991 — the system we’ll use hands-on later this semester.

NEXT WEEK

Definitions & Structures

Week 2 will cover:

- A precise, formal definition of an operating system
- The OS's design goals: convenience, efficiency, ability to evolve
- The four components of a computer system
- How operating systems are built: monolithic, layered, microkernel, modular

QUESTIONS?

Discussion

If you have questions after class, ask them on the course Telegram channel — everyone benefits from the answer.